Where Does the Cost of Bounds Checking Go? Dispatch-Aware Check Elimination and ISA-Preserving Loop-Guard Hoisting in a Stack-Based Virtual Machine

Abhiram
Department of Computer Science and Engineering (Data Science)
Vellore Institute of Technology, Vellore, India
[email]

***Abstract*—Array bounds checks are the canonical price of memory safety in virtual machines (VMs), and compile-time analysis is the canonical way to avoid paying it. Yet it is rarely made explicit *what* a check costs inside an interpreter: the comparison itself, or the extra bytecode that must be dispatched to perform it. We study this question on a complete, from-scratch toolchain: a small imperative language, a compiler with interval abstract interpretation, and a stack-based VM with computed-goto dispatch. The compiler emits five variants of the same program: unchecked (U), separate `CHK` instructions (R), fused checked loads/stores (F), interval-based elimination (S), and elimination plus a new *ISA-preserving loop-guard hoisting* pass (H) that versions innermost loops and expresses the guard with ordinary VM instructions, so the trusted VM is unchanged. On 13 benchmarks, separate checks cost $+10.0\%$ execution time (95% CI 9.2–11.0%) and this cost is almost entirely explained by the number of extra dispatched instructions (slope 0.93, $r = 0.90$); a fused check costs -0.5% (CI -1.3 to $+0.4$%), i.e., nothing measurable. Interval analysis removes 53.0% of dynamic checks and recovers 59% of the overhead; adding loop-guard hoisting removes 87.8% of dynamic checks and recovers 81%. A fault-injection study with 139 off-by-one mutants shows that the analysis raises new alarms for 91% of the 81 faulty mutants (with 3 false alarms among 58 benign ones), and differential testing found no behavioural divergence between any checked modes. We position the work as an applied, reproducible measurement study with two methodological contributions, not as new abstract-interpretation theory.**

***Index Terms*—bounds checking, virtual machines, abstract interpretation, check elimination, loop versioning, interpreter dispatch, fault injection**

I. Introduction

Memory-safe execution requires that every array access be proven, or checked, to lie within its object. Virtual machines for managed languages, smart-contract platforms and WebAssembly all enforce this, and the cost of doing so has been studied extensively in native and JIT-compiled settings [1]–[4]. Two families of remedies exist: make the check cheaper (hardware, guard pages, fat pointers [5], [6]), or make it unnecessary by proving it redundant at compile time [7]–[9].

Both remedies are usually evaluated against a native-code cost model in which a check is a compare and a well-predicted branch. In a bytecode *interpreter* the cost model is different: the dominant cost of an executed VM instruction is its dispatch [10], [11], so a check that is encoded as an extra instruction may be expensive for reasons unrelated to the comparison it performs. If so, the benefit of eliminating checks depends on how the checks are *encoded*, and cheap alternatives to elimination (fusing the check into the access opcode) deserve to be measured alongside it.

Research questions. We ask:

- **RQ1** What is the runtime overhead of bounds checking in a stack VM, and how much of it is due to dispatch rather than comparison?
- **RQ2** What fraction of checks can be eliminated at compile time, statically and dynamically, and how much overhead does this recover?
- **RQ3** Which classes of errors and accesses remain undetectable or unprovable by static analysis and must stay as runtime checks?

Contributions. Individual ingredients (interval analysis, check hoisting, loop versioning) are classical, and we claim no new abstract domain. Our contributions are:

1) *A dispatch-versus-comparison cost decomposition for VM bounds checks.* By compiling the same program to separate-check (R) and fused-check (F) encodings we isolate the cost of the comparison from the cost of the extra dispatch, and show that in our interpreter essentially all overhead is the latter (Section V).
2) *ISA-preserving loop-guard hoisting.* A compiler pass that versions innermost ascending loops and compiles the loop-entry guard into ordinary arithmetic and conditional-jump instructions. It needs no new VM opcode and no change to the trusted VM, yet removes checks that interval analysis alone cannot prove because the bound is a run-time parameter (Section III-E).

3) *A reproducible evaluation protocol* combining paired interleaved timing with bootstrap confidence intervals, a null-control benchmark that measures the noise floor, a mutation-based fault-injection study that classifies how each mode reacts to injected off-by-one faults, and differential testing across modes (Section IV).

The full toolchain, benchmarks and scripts are released with the paper.

II. Background and Related Work

Check elimination. Suzuki and Ishihata proposed one of the earliest array-bound checkers based on static assertions [7]. ABCD eliminates checks on demand using an inequality graph [8], and the HotSpot client compiler combines a fast analysis with check hoisting [9]. Our analysis is deliberately simpler than these (a non-relational interval domain with condition refinement), because our objective is to quantify the effect of elimination on a VM, not to improve the state of the art in precision. Richer relational domains such as polyhedra [12] and octagons [13] would prove strictly more, and mark the direction for future work.

Abstract interpretation. Intervals with widening and narrowing follow Cousot and Cousot [14]; we use them as in standard textbooks [15].

Cost of safety. WebAssembly's design relies on bounds checks and its overheads against native code have been analyzed in detail [1], [2], [16]. C-level spatial safety systems report overheads that depend on their metadata scheme [3]–[6], [17], [18]. In contrast to these native-code studies we measure a bytecode interpreter, where the encoding of the check interacts with dispatch.

VM design. Interpreter performance is governed by the number and cost of dispatched instructions [10], which is also the argument for register-based over stack-based VMs [11]. Our F variant can be read as the stack-VM analogue of a superinstruction that fuses a check into an access.

Compiler-assisted VM cost reduction. Static analysis has been used to reduce the runtime cost of bytecode in other VMs, e.g., gas optimization for Ethereum smart contracts [19]. Verified compilation [20] offers the strongest assurance that such transformations preserve safety; we instead validate preservation empirically by differential testing (Section IV).

III. System Design

A. Language and Toolchain

MiniL is a small imperative language with integer scalars, global fixed-size arrays, `if/else`, `while`, recursive functions and `print`. Fig. 1 shows the pipeline: lexical analysis, recursive-descent parsing to an AST, semantic analysis (name resolution, arity checks, local-slot allocation), static safety analysis (interval analysis and hoisting planner), and code generation parameterized by mode.

TABLE I
Compilation modes.

Mode	Check emission policy
U	No checks (fast, unsafe).
R	`CHK` before every `ALOAD`/`ASTORE` (explicit-check encoding).
F	Fused `ALOADC`/`ASTOREC` at every access (same checks, no extra dispatch).
S	As R, but omit `CHK` at sites proven safe by interval analysis.
H	As S, plus loop-guard hoisting (Section III-E).

B. Stack VM and Instruction Set

The VM executes 3-word instructions over an operand stack, a call stack and a flat memory in which arrays are laid out contiguously. The ISA comprises `PUSH`, `LOAD`/`STORE` (frame-relative), arithmetic and comparison operators, `JMP`/`JZ`/`JNZ`, `CALL`/`ENTER`/`RET`, `ALOAD`/`ASTORE` (array access), and three safety-related instructions: `CHK a`, which traps unless $0 \le$ `top` $< |a|$ (one unsigned comparison), and the fused `ALOADC`/`ASTOREC`, which perform `CHK` and the access in a single dispatch. Dispatch uses GCC computed goto. Memory is padded so that unchecked out-of-bounds accesses of moderate size corrupt neighbouring data rather than faulting immediately, as in real unchecked runtimes.

C. Five Compilation Modes

Table I lists the modes. U and R correspond to the plan's unchecked and runtime-checked modes, S and H to the compile-time-verified mode; F is an ablation we added to answer RQ1. A statically definite violation never changes the emitted code: it is reported as a compile-time diagnostic, and the access is still checked, so all checked modes trap identically.

D. Interval Analysis

For each function we run a forward abstract interpretation over the AST with the non-relational interval domain $\mathcal{I} = \{[l,h] \mid l \le h\} \cup \{\bot\}$ extended with $\pm\infty$, mapping each integer variable to an interval; parameters and values loaded from arrays are $\top$. Arithmetic uses standard interval transfer functions, with truncating division and C-style remainder. Branch and loop conditions *refine* the state: for a relation $x + k \bowtie e$ the interval of x is intersected with the interval of e shifted by $-k$, and symmetrically for the right operand, so that, e.g., $j < N-1-i$ with $i \in [0, N-2]$ yields $j \le N-3$. Loops are solved by Kleene iteration with widening after two iterations followed by two narrowing steps. A final recording pass joins, for every access site s into array a, the interval I_s of its index. Each site is classified:

$$\text{class}(s) = \begin{cases} \textsf{safe} & I_s \subseteq [0, |a|-1] \\ \textsf{violation} & I_s \cap [0, |a|-1] = \emptyset \\ \textsf{unknown} & \text{otherwise.} \end{cases}$$

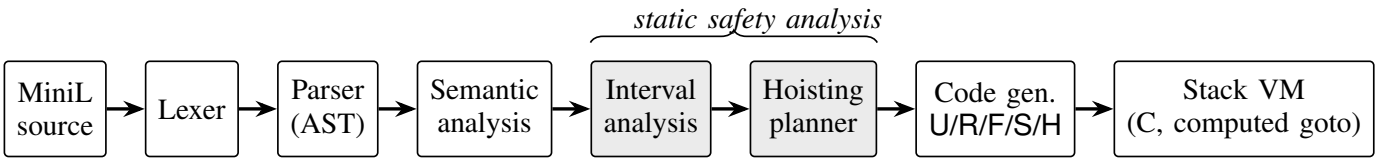


Fig. 1. Toolchain. Shaded stages implement the static safety analysis; only the code generator depends on the compilation mode.

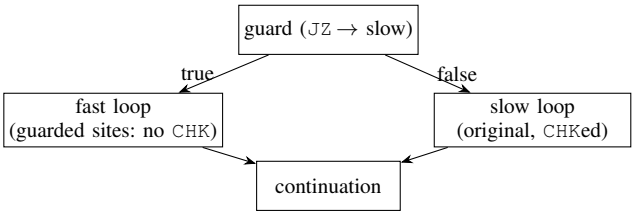


Fig. 2. Loop versioning with an ISA-level guard.

Under the assumption that integer arithmetic does not wrap, safe sites cannot go out of bounds in any execution and violation sites always do if reached (soundness follows from the standard argument [14]; we additionally test it empirically).

E. ISA-Preserving Loop-Guard Hoisting

Interval analysis cannot prove accesses whose bound is a parameter, e.g., `while (i < m) { s = s + a[i]; i = i + 1; }` with `m` unknown. Yet a single test before the loop suffices. The planner considers every innermost loop L of the form

$$\texttt{while}\ (i \lhd E)\,\{\ \ldots\ ;\ i = i + c\ \} \qquad \lhd \in \{<, \leq\},\ c \geq 1, \tag{1}$$

where the increment is the last statement and the only assignment to i, and E is a side-effect-free expression none of whose variables is assigned in the body. For every unknown site $a[i + K]$ in the body, with K side-effect-free and loop-invariant, it contributes the guard conjuncts

$$(i{+}K \geq 0) \wedge (E{+}d{+}K \leq |a|), \qquad d = \begin{cases} 1 & \lhd = \leq \\ 0 & \lhd = < \end{cases}. \tag{2}$$

Soundness. At every body execution, the loop condition holds and i has only increased since loop entry, so $i_0 \leq i \leq E{-}1{+}d$ (integers). Because i is modified only by the final statement, accesses see this range, hence $i + K \in [\, i_0 + K,\ E - 1 + d + K\,] \subseteq [0, |a| - 1]$ whenever the guard holds. □

Code shape. The guard is compiled with existing instructions. If it holds, execution enters a copy of the loop in which the guarded sites carry no check; otherwise it enters the original, fully checked loop (Fig. 2). Because the fallback is the original loop, a failing guard (including a spurious one for a loop that would not execute) preserves behaviour exactly, including the position and message of any trap. Since the transformation needs no new opcode, the trusted computing base of the VM is unchanged.

IV. Experimental Methodology

Benchmarks. Thirteen hand-written programs (Table II) in four groups: *statically provable* (`fill_sum`, `prefix`, `bubble`, `matmul`, `sieve`, `bsearch`), *parameter-bounded* (`param_sum`, `stencil_p`, `matvec_p`), *data-dependent* (`gather`, `hist`, `rec_sum`), and a *null control* (`fib`), which contains no array accesses and therefore compiles to identical bytecode in every mode. Any measured difference on `fib` is thus measurement noise or layout effect. Each program prints a checksum; all five modes produced identical output on every benchmark.

Platform and timing protocol. Programs run on a single virtual CPU of a Linux container (gcc `-O2`). Each (benchmark, mode) pair is run 25 times after one discarded warm-up round; modes are executed in randomly shuffled order inside each round, and the VM reports the wall-clock time of the interpreter loop only (process start-up excluded). Per-round ratios T_m/T_{U} are paired, we report their median and a 95% percentile-bootstrap interval (2000 resamples); aggregates are geometric means over the 12 benchmarks that access arrays. Instruction and check counts come from a separate instrumented build.

Fault injection. For each array benchmark (except the control) we generate mutants by (i) turning a loop condition `<` into `<=`, (ii) adding 1 to an index, (iii) subtracting 1 from an index, sampling up to five per operator, 139 in total. A mutant is *faulty* if mode R traps on it, and *benign* otherwise. We record whether static analysis (a) reports a definite violation, (b) raises more alarms (unknown+violation sites) than the unmutated program, or (c) neither, and how U behaves. We also compare the exit status, output and trap message of R, F, S and H on every mutant (differential testing).

V. Results

A. RQ1: Overhead of Checking and Its Dispatch Component

Separate runtime checks (R) cost $+10.0\%$ in geometric-mean execution time (95% CI 9.2–11.0%), ranging from $+1.2\%$ (`bsearch`) to $+21.5\%$ (`hist`) (Fig. 3, Table III). The number of executed VM instructions rises by the same 10.0% geometric mean. The null control shows that run-to-run and layout noise is of the order of $\pm 2\%$ (+0.7 to +1.7%), so differences of this size between modes on individual benchmarks should not be over-interpreted.

The fused encoding (F) performs exactly the same comparisons but adds no instructions: its geometric-mean overhead is -0.5% (CI -1.3 to $+0.4\%$), which is indistinguishable from zero. Fig. 4 makes the relation explicit: across the 13

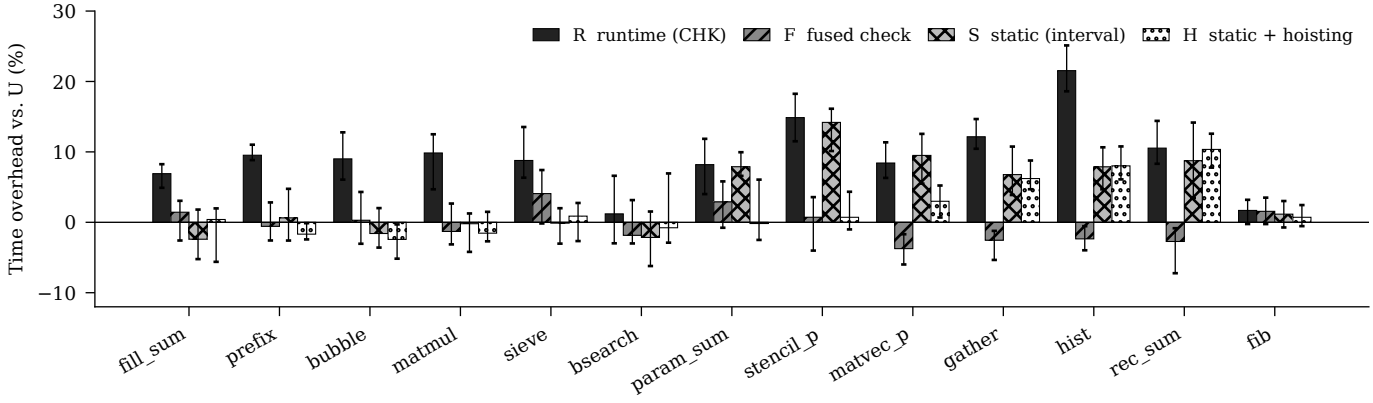


Fig. 3. Execution-time overhead relative to unchecked execution (U), with 95% bootstrap intervals over 25 paired rounds. Benchmarks are ordered from statically provable to data-dependent; `fib` is the null control.

TABLE II
BENCHMARKS: ACCESS SITES (STATIC), DYNAMIC CHECKS EXECUTED PER MODE, AND UNCHECKED INSTRUCTION COUNT.

	Static sites			Dynamic checks			Instr.
Benchmark	all	safe	hoist.	R	S	H	(M)
fill_sum	2	2	0	3 600 000	0	0	54.0
prefix	5	5	0	7 198 200	0	0	68.4
bubble	9	9	0	1 588 028	0	0	11.5
matmul	7	7	0	2 033 424	0	0	26.0
sieve	6	6	0	4 537 500	0	0	56.9
bsearch	2	2	0	937 340	0	0	27.4
param_sum	2	0	2	3 240 000	3 240 000	0	48.6
stencil_p	9	1	8	5 130 650	5 130 200	0	34.2
matvec_p	8	5	2	7 267 160	7 260 000	60 000	77.0
gather	4	3	0	4 004 000	2 000 000	2 000 000	30.1
hist	8	6	0	3 622 800	1 800 000	1 800 000	19.2
rec_sum	4	3	0	1 606 900	1 600 000	1 600 000	22.5
fib	0	0	0	0	0	0	29.6

TABLE III
OVERHEAD RELATIVE TO U (%). LEFT: EXECUTED-INSTRUCTION OVERHEAD. RIGHT: EXECUTION-TIME OVERHEAD (MEDIAN OF PAIRED RATIOS). THE `FIB` ROW HAS IDENTICAL BYTECODE IN ALL MODES, SO ITS TIME COLUMNS SHOW THE NOISE FLOOR.

	Instructions			Time			
Benchmark	R	S	H	R	F	S	H
fill_sum	+6.7	+0.0	+0.0	+6.9	+1.4	-2.4	+0.4
prefix	+10.5	+0.0	+0.0	+9.5	-0.6	+0.6	-1.7
bubble	+13.8	+0.0	+0.0	+9.0	+0.3	-1.6	-2.4
matmul	+7.8	+0.0	+0.0	+9.8	-1.3	-0.2	-1.5
sieve	+8.0	+0.0	+0.0	+8.8	+4.1	-0.1	+0.9
bsearch	+3.4	+0.0	+0.0	+1.2	-1.8	-2.1	-0.8
param_sum	+6.7	+6.7	+0.1	+8.2	+2.9	+7.9	-0.2
stencil_p	+15.0	+15.0	+0.1	+14.9	+0.7	+14.2	+0.7
matvec_p	+9.4	+9.4	+2.6	+8.4	-3.7	+9.5	+3.0
gather	+13.3	+6.7	+6.7	+12.1	-2.5	+6.8	+6.2
hist	+18.9	+9.4	+9.4	+21.5	-2.4	+7.9	+8.0
rec_sum	+7.1	+7.1	+7.1	+10.5	-2.7	+8.7	+10.4
fib	+0.0	+0.0	+0.0	+1.7	+1.6	+1.2	+0.7

benchmarks the time overhead of R follows the fraction of extra dispatched instructions with slope 0.93 and Pearson $r = 0.90$ (dotted line: $y = x$), while F points sit at zero extra instructions with time overhead scattered around zero. In this interpreter, *the cost of a bounds check is the cost of dispatching it*, not the cost of comparing.

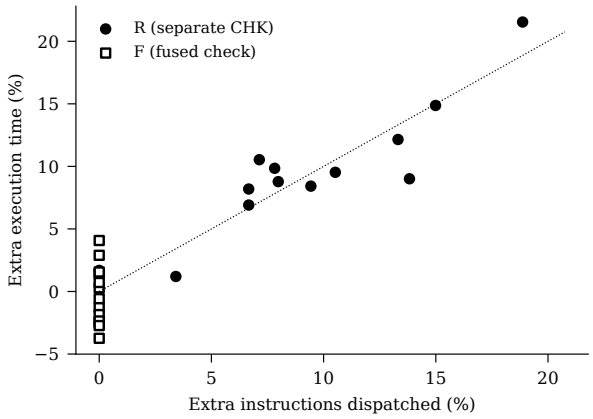


Fig. 4. Time overhead versus extra instructions dispatched. Separate checks (R) track $y = x$; fused checks (F) add no instructions and no measurable time.

B. RQ2: Static and Dynamic Check Elimination

Over the 12 array benchmarks there are 66 access sites. Interval analysis proves 49 safe (74.2%); hoisting guards a further 12 (18.2%); 5 sites (7.6%) remain as residual runtime checks (Fig. 5a). Static counts understate the benefit, because the proved sites tend to lie in hot loops: dynamically, R executes 44.8 M checks, S 21.0 M (53.0% eliminated) and H 5.5 M (87.8% eliminated) (Fig. 5b).

Translating elimination into time, S reduces the geometric-mean overhead from $+10.0\%$ to $+4.0\%$ (CI 3.0–4.9%), recovering 59% of R's overhead (55% by instruction count); H reaches $+1.8\%$ (CI 1.1–3.1%), recovering 81% (79% by instruction count), which is close to the $\pm 2\%$ noise floor. Hoisting is decisive precisely where interval analysis fails on a parameter-bounded loop: `param_sum` (S $+7.9\%$, H -0.2%) and `stencil_p` ($+14.2\% \rightarrow +0.7\%$). It adds nothing on `gather`, `hist` and `rec_sum`, whose indices are loaded data or recursion parameters. Hoisting costs code size: bytecode

grows by $1.16\times$ (geometric mean) for H versus $1.05\times$ for R and $1.01\times$ for S, as loops with guarded sites are duplicated.

C. RQ3: What Remains Undetectable Statically

The five residual sites fall into three classes. (i) *Data-dependent indices*: values loaded from memory are $\top$ in a non-relational domain, e.g., `a[p[i]]` in `gather` and `h[d[i]%64]` in `hist` (the remainder of an unknown signed value is only bounded by ± 63). (ii) *Recursion parameters*: `a[i]` in `rec_sum`, where i is a parameter and the analysis is intraprocedural. (iii) *Non-innermost parameter loops*: `y[i]` in `matvec_p`, because we version innermost loops only. Classes (ii) and (iii) are limits of our implementation rather than of static analysis, whereas (i) calls for relational or value-set reasoning and is inherently limited when the data come from the environment.

Fault injection. Of 139 mutants, 81 (58.3%) are faulty and 58 benign (Fig. 6). Static analysis reports a *definite* compile-time violation for only 4 faulty mutants (4.9%): an off-by-one error typically makes the index interval *straddle* the array bound rather than lie wholly outside it, which the analysis can only flag as unknown. It nevertheless raises strictly more alarms than the original program for 70 further faulty mutants, so 74 of 81 (91.4%) are noticed statically, versus 3 of 58 benign mutants (5.2%) that raise a false alarm (alarm-delta precision 96.1%). The 7 missed faulty mutants mutate a site that was already unknown, so no new alarm can appear. Under U, *none* of the 81 faulty mutants produced a diagnostic or crash: all ran to completion silently. This is the safety gap that motivates checking at all.

Differential testing. For all 139 mutants, R, F, S and H agreed on exit status, standard output and trap message; no mode trapped where another did not, and no safe or guarded access ever trapped. On the 13 original benchmarks all modes produced identical output. This is empirical evidence, not proof, that elimination and hoisting preserve the semantics of checked execution.

VI. Discussion and Threats to Validity

When does elimination matter? Our results suggest a design rule for interpreters: before investing in analysis, fuse the check into the access instruction. This removes essentially all overhead here (-0.5%) at no analysis cost. Static elimination remains valuable when checks are encoded as separate instructions (as in explicit-check bytecodes), in JIT/AOT code where a compare-and-branch is not free, and for *diagnostics*, where its value is independent of speed. We do not claim that fused checks are free on all hardware or that our $\approx$10% figure generalizes to native code, where studies report a wide range of overheads [2].

Internal validity. Timing is on a single virtual CPU with $\sim$2% noise (control benchmark), so conclusions about differences below that level, including the comparison of H with U, are not supported. Interval soundness assumes no integer wrap-around (not modelled), and the hoisting soundness argument is a proof sketch, supplemented by differential testing rather than mechanized verification.

External validity. The benchmark set is small (13 programs) and hand-written, the language has only global fixed-size arrays, and the VM is a simple computed-goto interpreter; a JIT, registers, or heap objects with dynamic lengths would change both costs and analysis. Mutants are synthetic faults and are not a substitute for field bug data.

Novelty scope. Range-check hoisting by loop versioning is known in optimizing compilers. What we add is (i) its expression purely in the VM's existing ISA so the VM need not change, (ii) its quantification on top of interval elimination in an interpreter setting, and (iii) the decomposition showing the dispatch effect. We are not aware of a study that combines these three in a single controlled setting, but we have not performed an exhaustive literature search.

VII. Conclusion and Future Work

In a bytecode interpreter, the bounds-check overhead of $+10\%$ is a dispatch cost: encoding the check in the access opcode removes it, and ISA-preserving hoisting plus interval analysis removes 88% of dynamic checks and recovers 81% of the overhead of the explicit encoding while keeping the VM unchanged. A fault-injection study quantifies the residual role of runtime checks: data-dependent indices and recursion remain, and off-by-one faults are mostly detectable as alarms but rarely as definite errors. Future work includes relational domains, interprocedural summaries, hoisting for non-innermost and descending loops, register-based VMs, JIT back-ends, and larger benchmark suites.

References

[1] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien, "Bringing the web up to speed with WebAssembly," in *Proc. 38th ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

[2] A. Jangda, B. Powers, E. D. Berger, and A. Guha, "Not so fast: Analyzing the performance of WebAssembly vs. native code," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 107–120.

[3] S. Nagarakatte, J. Zhao, M. M. K. Martin, and S. Zdancewic, "SoftBound: Highly compatible and complete spatial memory safety for C," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2009, pp. 245–258.

[4] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2012, pp. 309–318.

[5] P. Akritidis, M. Costa, M. Castro, and S. Hand, "Baggy bounds checking: An efficient and backwards-compatible defense against out-of-bounds errors," in *Proc. 18th USENIX Security Symp.*, 2009, pp. 51–66.

[6] G. C. Necula, J. Condit, M. Harren, S. McPeak, and W. Weimer, "CCured: Type-safe retrofitting of legacy software," *ACM Transactions on Programming Languages and Systems*, vol. 27, no. 3, pp. 477–526, 2005.

[7] N. Suzuki and K. Ishihata, "Implementation of an array bound checker," in *Proc. 4th ACM SIGACT-SIGPLAN Symp. Principles of Programming Languages (POPL)*, 1977, pp. 132–143.

[8] R. Bodík, R. Gupta, and V. Sarkar, "ABCD: Eliminating array bounds checks on demand," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2000, pp. 321–333.

[9] T. Würthinger, C. Wimmer, and H. Mössenböck, "Array bounds check elimination for the Java HotSpot client compiler," in *Proc. 5th Int. Symp. Principles and Practice of Programming in Java (PPPJ)*, 2007, pp. 125–133.

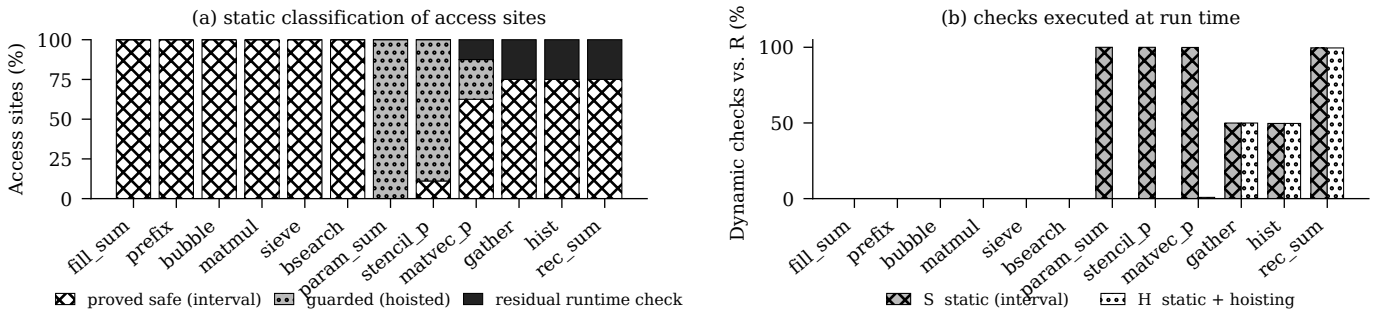


Fig. 5. (a) Static classification of access sites; (b) dynamic checks executed by S and H as a percentage of those executed by R. Benchmarks with fully provable sites show 0% in (b).

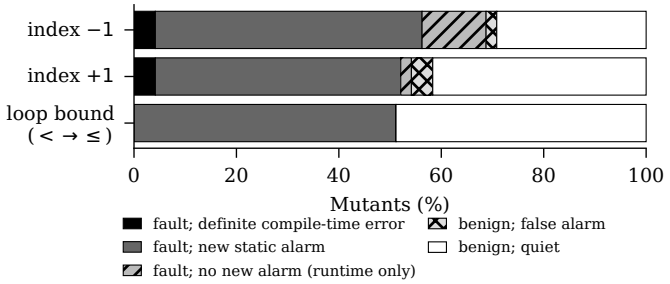


Fig. 6. Outcome of 139 injected off-by-one faults, by mutation operator. "Alarm" means the number of non-proved sites exceeds that of the unmutated program.

[10] M. A. Ertl and D. Gregg, "The structure and performance of efficient interpreters," in *Journal of Instruction-Level Parallelism*, vol. 5, 2003.

[11] Y. Shi, K. Casey, M. A. Ertl, and D. Gregg, "Virtual machine showdown: Stack versus registers," *ACM Transactions on Architecture and Code Optimization*, vol. 4, no. 4, p. Article 2, 2008.

[12] P. Cousot and N. Halbwachs, "Automatic discovery of linear restraints among variables of a program," in *Proc. 5th ACM Symp. Principles of Programming Languages (POPL)*, 1978, pp. 84–96.

[13] A. Miné, "The octagon abstract domain," *Higher-Order and Symbolic Computation*, vol. 19, no. 1, pp. 31–100, 2006.

[14] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM Symp. Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[15] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.

[16] A. Rossberg, B. L. Titzer, A. Haas, D. L. Schuff, D. Gohman, L. Wagner, A. Zakai, J. F. Bastien, and M. Holman, "Bringing the web up to speed with WebAssembly," *Communications of the ACM*, vol. 61, no. 12, pp. 107–115, 2018.

[17] T. M. Austin, S. E. Breach, and G. S. Sohi, "Efficient detection of all pointer and array access errors," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 1994, pp. 290–301.

[18] R. W. M. Jones and P. H. J. Kelly, "Backwards-compatible bounds checking for arrays and pointers in C programs," in *Proc. 3rd Int. Workshop on Automated Debugging (AADEBUG)*, 1997, pp. 13–26.

[19] E. Albert, P. Gordillo, A. Rubio, and I. Sergey, "GASOL: Gas analysis and optimization for Ethereum smart contracts," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2020.

[20] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.